

K-Means Clustering

Aim:

To group similar data points into clusters using the K-Means Clustering algorithm in Python and visualize the clusters.

Procedure:

1. Import Libraries: Import necessary libraries like pandas, numpy, matplotlib.pyplot, and seaborn.
2. Load Dataset: Load the dataset containing the features to be clustered.
3. Select Features: Choose the relevant numeric columns for clustering.
4. Create K-Means Model: Initialize the K-Means model with a specific number of clusters (n_clusters).
5. Fit the Model: Apply the K-Means algorithm to the dataset to compute cluster centroids.
6. Assign Cluster Labels: Predict and assign each data point to a cluster.
7. Add Labels to DataFrame: Store the cluster labels in the dataset for reference.
8. Visualize Clusters: Use scatterplots or Seaborn's FacetGrid to visualize the clusters with different colors for each cluster.

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
df=pd.read_csv('Mall_Cust.csv')
df.head()
```

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)	
0	1	Male	19	15	39	
1	2	Male	21	15	81	
2	3	Female	20	16	6	
3	4	Female	23	16	77	
4	5	Female	31	17	40	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   CustomerID                            200 non-null    int64
1   Gender                                200 non-null    object
2   Age                                    200 non-null    int64
3   Annual Income (k$)                    200 non-null    int64
4   Spending Score (1-100)                200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

```
df.isnull().sum()
```

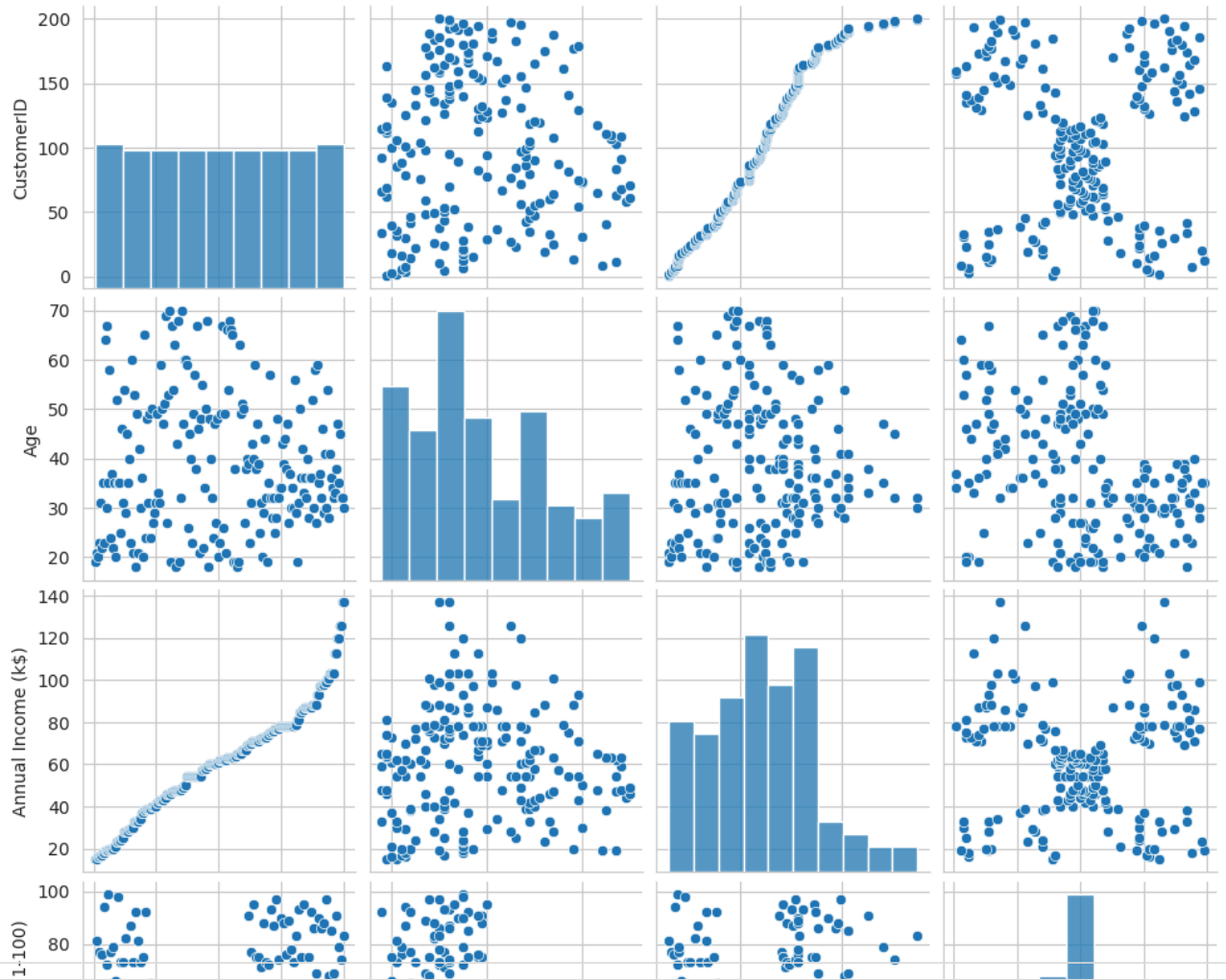
```

      0
CustomerID    0
Gender        0
Age           0
Annual Income (k$)  0
Spending Score (1-100)  0
```

```
dtype: int64
```

```
plt.figure(figsize=(3,1))
sns.pairplot(df)
plt.show()
```

<Figure size 300x100 with 0 Axes>



```
features=df.iloc[:,[3,4]].values
```

```
from sklearn.cluster import KMeans
model=KMeans(n_clusters=5)
model.fit(features)
```

0 50 100 150 200 20 40 60 50 100 0 25 50 75 100
 ID Age Annual Income (k\$) Spending Score (1-100)
 KMeans (i ?)
 KMeans(n_clusters=5)

```
Final=df.iloc[:,[3,4]]
Final['label']=model.predict(features)
Final.head()
```

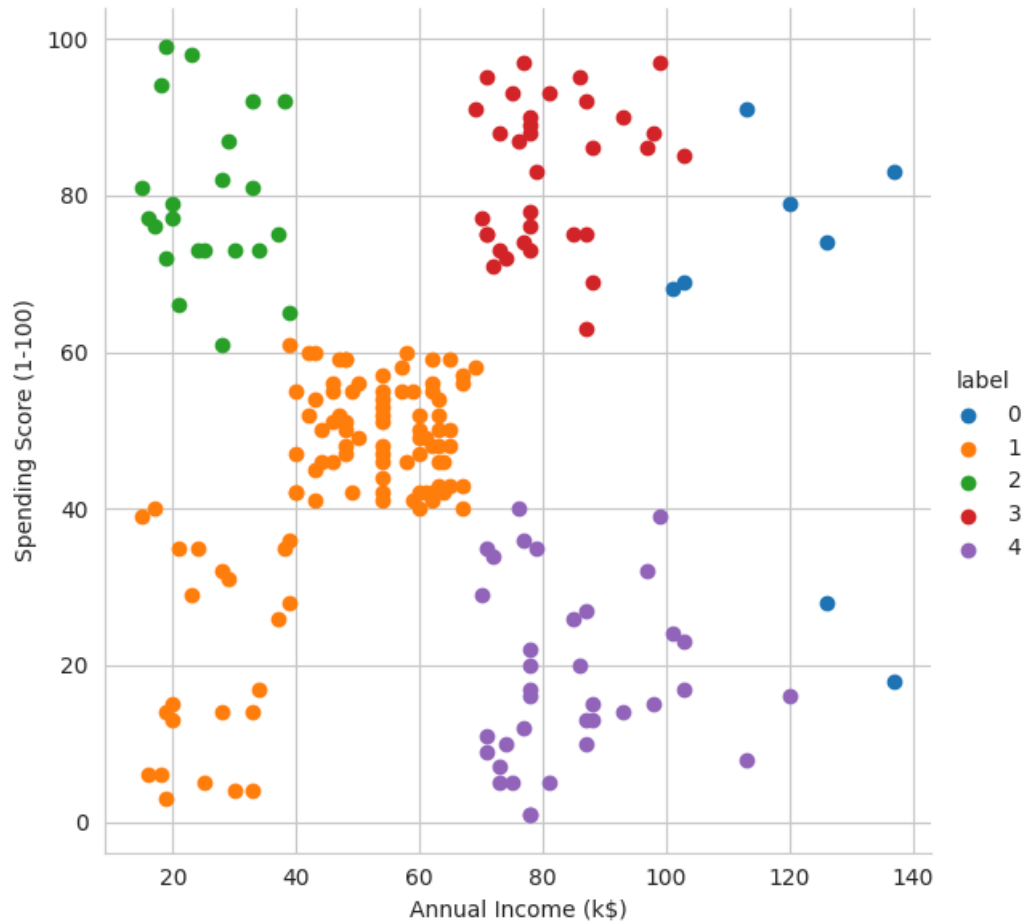
/tmp/ipython-input-470183701.py:2: SettingWithCopyWarning:
 A value is trying to be set on a copy of a slice from a DataFrame.
 Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html
 Final['label']=model.predict(features)

	Annual Income (k\$)	Spending Score (1-100)	label
0	15	39	1
1	15	81	2
2	16	6	1
3	16	77	2
4	17	40	1

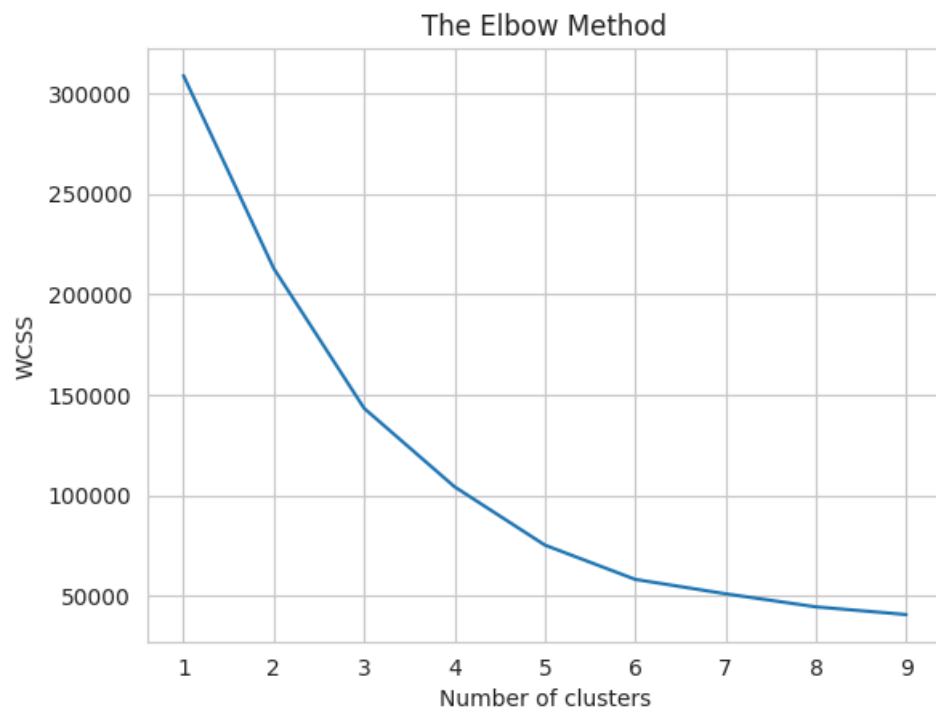
Next steps: [Generate code with Final](#)[New interactive sheet](#)

```
sns.set_style("whitegrid")
sns.FacetGrid(Final,hue="label",height=6) \
.map(plt.scatter,"Annual Income (k$)", "Spending Score (1-100)") \
.add_legend();
plt.show()
```



```
features_el=df.iloc[:,[2,3,4]].values
from sklearn.cluster import KMeans
wcss=[]
for i in range(1,10):
    model=KMeans(n_clusters=i, init='k-means++', random_state=42, n_init=10)
    model.fit(features_el)
    wcss.append(model.inertia_)

plt.plot(range(1,10),wcss)
plt.title('The Elbow Method')
plt.xlabel('Number of clusters')
plt.ylabel('WCSS')
plt.show()
```



Result:

Hence, a Python program to perform K-Means clustering has been implemented successfully, grouping the data into distinct clusters.